Empirical Analysis of Insertion Sort

CS314 Analysis of Algorithms · Lecture 1 mini project

1. Problem

Given a sequence of n numbers, produce a permutation of it in non-decreasing order. This project studies insertion sort empirically: it implements the algorithm in C++, counts element comparisons and shifts, times it for n = 100 to 5,000 on sorted, reverse-sorted and random inputs, and compares the measured growth with the $O(n)$ and $O(n^2)$ predictions and with merge sort. The question is whether the measurements match the analysis, and when insertion sort is the right choice.

2. Algorithm and Complexity

Insertion sort keeps $A[1..j-1]$ sorted. For each key $A[j]$ it shifts every larger element one position to the right and inserts the key in the gap. It sorts in place, so it needs $O(1)$ extra memory. Merge sort, the baseline, splits the array in halves, sorts each half and merges them: $\Theta(n \log n)$ time in every case and $O(n)$ extra memory.

Let $t(j)$ be the number of times the while test runs for key j. The running time is the sum of the statement costs, and it depends only on the $t(j)$:

- **Best case (sorted):** $t(j) = 1$, so $n - 1$ comparisons and no shifts. $T(n) = an + b$, which is $\Omega(n)$.
- **Worst case (reverse sorted):** $t(j) = j$, so $n(n-1)/2$ comparisons and $n(n-1)/2$ shifts. $T(n) = an^2 + bn + c$, which is $O(n^2)$.
- **Average case (random):** each key passes about half of the sorted part, so about $n^2/4$ shifts. The time is $\Theta(n^2)$.

The counters count only comparisons between two elements: for [7, 3, 9, 2, 5] they give 8 comparisons and 6 shifts, as in the lecture's worked example. Quadratic growth predicts that 5× more elements costs about 25× more work.

3. Experiments

The code is C++17 compiled with g++ -O2. Sizes are n = 100, 500, 1,000, 1,500, …, 5,000. Inputs are a sorted array, a reverse-sorted array and a random permutation of 1..n (fixed seed 42). Each time is the median of 21 runs on fresh copies of the input; operation counts

are averages over the same runs. Every run checks that the output is sorted.

Table 1. Operation counts and median running time.

Input	n	Insertion comparisons	Insertion shifts	Insertion time (µs)	Merge time (µs)
Sorted	1,000	999	0	1.14	15.2
Sorted	5,000	4,999	0	5.51	81.8
Reversed	1,000	499,500	499,500	242.9	15.5
Reversed	5,000	12,497,500	12,497,500	5,916	92.6
Random	1,000	251,305	250,312	128.2	64.3
Random	5,000	6,251,099	6,246,108	2,995	386.5

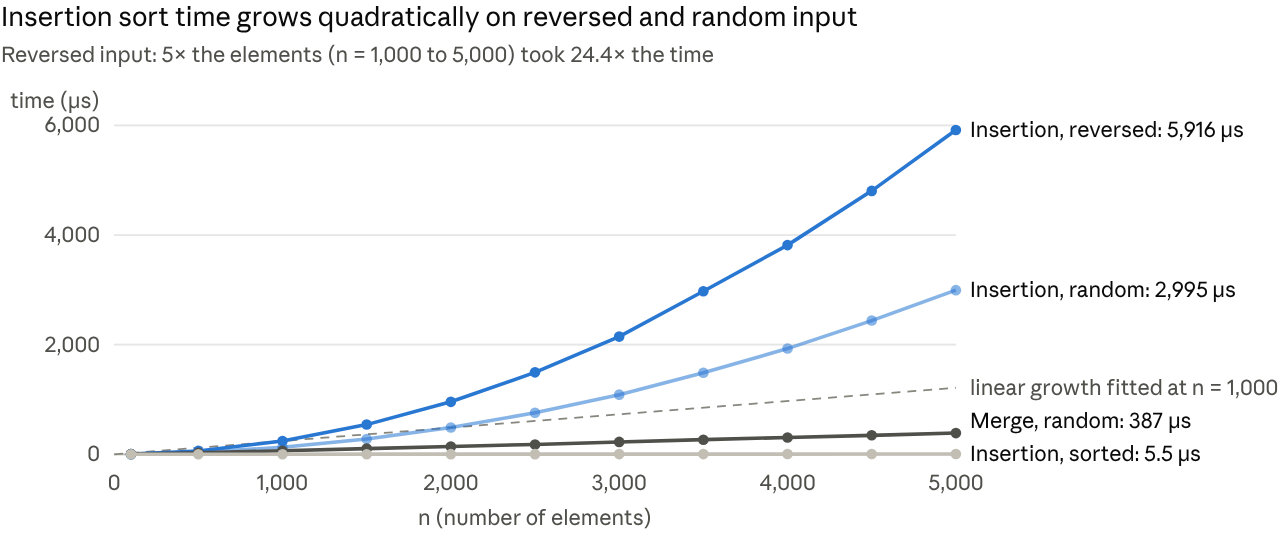


Figure 1. Median of 21 runs per point, from results/results.csv

The counts match the analysis exactly: $n - 1$ comparisons on sorted input, $n(n-1)/2 = 12{,}497{,}500$ on reversed input at $n = 5{,}000$, and about $n^2/4$ on random input. The time follows the counts. From $n = 1{,}000$ to $5{,}000$ the reversed input needs 25× the comparisons and 24× the time, while the sorted input needs 4.8× the time. A least-squares fit of log time against log n gives slopes of 0.94 for sorted, 1.94 for reversed and 1.82 for random input, close to the predicted 1 and 2. The random slope is lower probably because fixed costs and branch mispredictions weigh more at small n.

Merge sort grows like $n \log n$ (slopes 1.08 to 1.13). At $n = 5{,}000$ it is 7.7× faster than insertion sort on random input and 64× faster on reversed input. On sorted input insertion sort is 15× faster, because it makes a single pass of $n - 1$ comparisons. On random input

the two cross between n = 100 (insertion 2.7 µs, merge 4.8 µs) and n = 500 (insertion 34.5 µs, merge 29.3 µs).

4. Conclusion

The experiments confirm the analysis: insertion sort is Ω(n) on sorted input and quadratic on reversed and random input, and its cost is driven by the number of shifts. It needs no extra memory and has a small constant, so it is a good choice for small inputs and for inputs that are sorted or nearly sorted. For large random data its quadratic growth makes merge sort the better choice, at the price of O(n) extra memory: the time–space trade-off. Absolute times come from one machine and will differ elsewhere, but the operation counts and growth rates do not.